Informe de Desarrollo — Módulo Eventos Programados

Sistema Especializado de Proyectos — GGPC SENA Fecha: Abril 2026 | **Estado:** Implementado

1. Descripción General

El módulo de Eventos Programados muestra el listado de eventos activos del GGPC en las diferentes convocatorias vigentes. Es público (no requiere autenticación). Permite a los usuarios externos visualizar el estado de cada evento y registrarse en los que estén activos y visibles, mediante un wizard de registro multi-paso.

2. Flujo General

Usuario accede a /eventos

```
    |
    ▼
```

GET /api/eventos → Lista de eventos desde Oracle

```
    |
    ▼
```

Tabla con: Nombre | Fecha Inicio | Fecha Fin | Visibilidad | Estado | Registrar

```
    |
    ▼
```

Si evento ACTIVO + VISIBLE → botón "Registrar"

```
    |
    ▼
```

/eventos/:id/registrar → Wizard de registro (5 pasos)

3. Frontend

Archivos principales

Archivo	Rol
frontend/src/app/(public)/eventos/page.tsx	Página Server Component — título y contenedor
frontend/src/components/public/eventos/eventos-list.tsx	Listado dinámico de eventos (Client Component)
frontend/src/app/(public)/eventos/[id]/registrar/page.tsx	Página de registro a un evento
frontend/src/components/public/eventos/registro/registro-wizard.tsx	Wizard de registro multi-paso
frontend/src/components/public/eventos/registro/step-buscar-persona.tsx	Paso 1: buscar persona por documento
frontend/src/components/public/eventos/registro/step-datos-basicos.tsx	Paso 2: datos básicos
frontend/src/components/public/eventos/registro/step-datos-empresa.tsx	Paso 3: datos de la empresa
frontend/src/components/public/eventos/registro/step-habeas-data.tsx	Paso 4: Habeas Data
frontend/src/components/public/eventos/registro/step-confirmar.tsx	Paso 5: confirmación final

Listado de eventos (EventosList)

- Client Component con useEffect que llama GET /api/eventos.

- Muestra spinner durante la carga.
- Tabla con columnas: Nombre, Fecha Inicio, Fecha Fin, Visibilidad, Estado, Registrar.
- **Badges de estado:**
    - ACTIVO / INACTIVO → verde o gris
    - VISIBLE / NO VISIBLE → verde o gris
- Botón "Registrar" solo aparece si el evento es simultáneamente ACTIVO y VISIBLE.

```
// Solo eventos activos Y visibles muestran el botón
{ev.eventoActivo && ev.eventoVisible ? (
  <a href={`/eventos/${ev.eventoId}/registrar`}>Registrar</a>
) : (
  <span>—</span>
)}
```

Wizard de Registro (5 pasos)

Paso 1 — Buscar Persona

- El usuario ingresa tipo de documento y número de identificación.
- Llama GET /api/auth/tipos-documento?para=persona para el dropdown de tipos.
- Llama GET /api/eventos/:id/persona?tipoDoc=X&numero=Y para buscar si la persona ya existe.
- Si existe → precarga sus datos en el wizard.
- Si no existe → permite registrar persona nueva.

Paso 2 — Datos Básicos

- Nombres, apellidos, género, fecha de nacimiento.
- Datos prellenados si la persona fue encontrada en el paso 1.

Paso 3 — Datos de la Empresa

- Empresa donde labora el participante.
- Cargo, sector.

Paso 4 — Habeas Data

- Presentación del texto legal de tratamiento de datos personales.
- Checkbox obligatorio de aceptación antes de continuar.

Paso 5 — Confirmación

- Resumen de los datos ingresados.
- Botón "Confirmar registro" → POST /api/eventos/:id/registrar.
- Toast de éxito con número de registro asignado.

Componentes de UI utilizados

Componente	Uso en el módulo
BadgeEstado	Badges de Activo/Visible en la tabla
<Loader2> (Lucide)	Spinner de carga del listado
<CalendarX> (Lucide)	Estado vacío sin eventos
<AlertCircle> (Lucide)	Estado de error de la API
<ClipboardCheck> (Lucide)	Botón registrar

---

4. Backend

Archivos involucrados

Archivo	Rol
backend/src/eventos/eventos.controller.ts	Endpoints de eventos
backend/src/eventos/eventos.service.ts	Lógica de consulta y registro

Endpoints

Listar eventos:

GET /eventos

→ Array de eventos activos y visibles

Registro en evento:

POST /eventos/:id/registrar

Body: { personaId?, nombres, apellidos, tipoDoc, numero, ... }

Query principal de eventos

```
SELECT E.EVENTOID, TRIM(E.EVENTONOMBRE) AS "eventoNombre",
       E.EVENTOFECHAINICIO AS "eventoFechaInicio",
       E.EVENTOFECHAFIN AS "eventoFechaFin",
       CASE WHEN TRIM(E.EVENTOVISIBLE) = 'SI' THEN 1 ELSE 0 END AS "eventoVisible",
       CASE WHEN TRIM(E.EVENTOACTIVO) = 'SI' THEN 1 ELSE 0 END AS "eventoActivo"
FROM EVENTO E
WHERE TRIM(E.EVENTOACTIVO) = 'SI'
ORDER BY E.EVENTOFECHAINICIO DESC
```

Tipo Evento (TypeScript)

```
// frontend/src/types/index.ts
interface Evento {
  eventoId: number
  eventoNombre: string
  eventoFechaInicio: string
  eventoFechaFin: string
  eventoVisible: boolean
  eventoActivo: boolean
}
```

Captura Página de eventos: